# Clarifying Requirements & Trade-offs

## 1. Overview

This is the **first and most critical step** in any system design interview.

> If you get this wrong → entire design can go in the wrong direction

## 2. Time Strategy

- Spend **5–10 minutes max**
- Don't go too deep
- Don't skip it

Balance:

- Enough clarity
- Not over-discussing

## 3. Step 1: Understand the Big Picture (30–60 sec)

Start with:

> "Let me first understand the overall goal of the system."

Clarify:

- What problem are we solving?
- Who are the users?
- What's the business goal?

# 4. Step 2: Functional Requirements

Use **brainstorm → then discuss approach**

### 1. Identify Users & Roles

Ask:

- Who are the users?
- What are their roles?

Examples:

- Buyer / Seller
- Creator / Viewer
- Admin / Developer

### 2. User Stories

Write quickly:

- User can create post
- User can view feed
- User can search

Think in terms of:

- Actions
- Use cases

### 3. User Types

- Consumer vs Enterprise
- Manual vs Programmatic (APIs)
- Technical vs Non-technical

### 4. Add Numbers

Every requirement should have numbers

Instead of:
“Fetch posts”

Say:
“Fetch top 20 posts within 200ms”

### 5. Communication Needs

- User ↔ User (chat, comments)
- User ↔ System (notifications)

### 6. Internationalization (i18n) / Localization (l10n)

Ask:

- Multi-language support?
- Currency support?
- Regional formats?

### 7. Authentication & Authorization

- Who can access what?
- Roles & permissions

### 8. Data Access Patterns

- Real-time vs batch
- Read-heavy vs write-heavy

### 9. Search Requirements

- Full-text search?
- Filters?
- Ranking?

### 10. Analytics & ML

- Metrics tracking
- A/B testing
- Recommendation systems

### 11. API Thinking

Write quick signatures:

```python
- getPosts(userId)
- createPost(userId, content)
- searchPosts(query)
```

Shows practical thinking

## 5. Step 3: Scalability Estimation

Convert users → system load

Example:

- 1M users
- Each makes 10 requests/day

→ 10M requests/day

**Estimate:**

- QPS
- Storage
- Bandwidth

This drives architecture decisions

## 6. Step 4: Data Access & Security

Ask:

- Which user can access which data?
- Is data public/private?
- Any compliance requirements?

## 7. Step 5: Non-Functional Requirements

Must include:

- Latency (e.g., <200ms)

- Availability (e.g., 99.9%)
- Consistency
- Scalability
- Reliability

## 8. Step 6: Trade-offs (Always Mention)

Every decision has trade-offs

Examples:

- Consistency vs Availability
- Latency vs Accuracy
- Cost vs Performance

## 9. Step 7: Ask Open-Ended Questions

Always ask:

> “Are there any additional requirements or constraints I should consider?”

Shows:

- Curiosity
- Awareness

## 10. Step 8: Think Ahead

Mention:

> “This system can be extended in future to support…”

Examples:

- Recommendations
- Notifications

- Analytics

## 11. Common Mistakes

- Jumping into design too early
- Not adding numbers
- Ignoring user roles
- Missing non-functional requirements
- Not asking clarifying questions

## 12. Perfect Interview Script

You can say:

> "I'll first clarify the functional and non-functional requirements. I'll identify user roles, define key use cases, estimate scale in terms of QPS, and confirm latency and availability requirements. Then I'll proceed to design the system."

This sounds **very professional**

## 13. Mental Model

Think like:

**Product Manager + Engineer**

- What users want
- How system handles it

## 14. Final Summary

- Start with big picture

- Identify users & use cases
- Add numbers to requirements
- Estimate scale (QPS)
- Discuss security & access
- Include non-functional requirements
- Always discuss trade-offs